PROGRMA

Biblioteca Smart

Programación Avanzada

Alumna:

ESTEFANIA MELÉNDEZ ATONAL

matrícula: 202524453

Clase Material:

```python
from datetime import datetime, timedelta

class Material ():
    def __init__ (self, idMaterial, titulo, añoPublicacion, disponible):
        self.idMaterial= idMaterial
        self.titulo= titulo
        self.añoPublicacion= añoPublicacion
        self.disponible= disponible

    def actualizarDisponibilidad(self):
        self.disponible = not self.disponible
        print(f"Disponibilidad de '{self.titulo}' actualizada a: {self.disponible}")

    def obtenerDetalles(self):
        print(f"ID: {self.idMaterial}  //  Título: {self.titulo} //  Año: {self.añoPublicacion}")
```

Objetos:

```python
print("\n""----Biblioteca Smart----""\n")
mat1= Material(1244556, "El principito", 1943, True)
mat2 = Material(1244557, "Crónica de una muerte anunciada", 1981, True)
mat3 = Material(1244558, "Fahrenheit 451", 1953, False)
mat4 = Material(1244559, "El Aleph", 1949, True)
mat5 = Material(1244560, "La ciudad y los perros", 1963, True)
mat6 = Material(1244561, "Ensayo sobre la ceguera", 1995, False)
mat7 = Material(1244562, "Rayuela", 1963, True)
mat8 = Material(1244563, "Drácula", 1897, True)
mat9 = Material(1244564, "Moby Dick", 1851, False)
mat10 = Material(1244565, "Frankenstein", 1818, True)
```

Uso de métodos:

```
mat1.actualizarDisponibilidad()
print("\n")
mat1.obtenerDetalles()
```

Terminal:

```
----Biblioteca Smart----

Disponibilidad de 'El principito' actualizada a: False


ID: 1244556  //  Título: El principito //  Año: 1943
```

Clase Libro:

```
class Libro(Material):
    def __init__(self, autor, isbn, genero, idMaterial, titulo, añoPublicacion, disponible ):
        super(). __init__(idMaterial, titulo, añoPublicacion, disponible)
        self.autor= autor
        self.isbn= str(isbn)
        self.genero= genero

    def validarISBN(self):
        if len(self.isbn) == 13:
            print(f"ISBN verificado: {self.isbn}")
        else:
            print("ISBN inválido, verifica los dígitos")

    def consultarAutor(self):
        print(f"El autor de {self.titulo} es {self.autor}")
```

Objetos:

```
book1= Libro( "Dan Brown", "1894826136785", "Thriller", 1731183, "Inferno", 2013, True )
book2 = Libro("Gabriel García Márquez", "9780307474728", "Realismo Mágico", 1731184, "Cien años de soledad", 1967, True)
book3 = Libro("George Orwell", "9780451524935", "Distopía", 1731185, "1984", 1949, False)
book4 = Libro("J.K. Rowling", "9780747532743", "Fantasía", 1731186, "Harry Potter y la piedra filosofal", 1997, True)
book5 = Libro("Isabel Allende", "97800608", "Ficción Histórica", 1731187, "La casa de los espíritus", 1982, True)
book6 = Libro("Frank Herbert", "9780441172719", "Ciencia Ficción", 1731188, "Dune", 1965, False)
book7 = Libro("Stephen King", "9781501142970", "Terror", 1731189, "It (Eso)", 1986, True)
book8 = Libro("Antoine", "9780156012195", "Infantil", 1731190, "El Principito", 1943, True)
book9 = Libro("Agatha Christie", "9780062073402", "Misterio", 1731191, "Asesinato en el Orient Express", 1934, True)
book10 = Libro("Ernest Hemingway", "978068480", "Aventura", 1731192, "El viejo y el mar", 1952, False)
```

Uso de métodos:

```
book6.validarISBN()
print("\n")
book5.validarISBN()
print("\n")
book3.consultarAutor()
```

Terminal:

```
----Biblioteca Smart----

ISBN verificado: 9780441172719


ISBN inválido, verifica los dígitos


El autor de 1984 es George Orwell
PS C:\Users\LATITUDE 5550>
```

Clase Revista:

```python
class Revista(Material):
    def __init__( self, edicion, periodicidad, idMaterial, titulo, añoPublicacion, disponible):
        super(). __init__(idMaterial, titulo, añoPublicacion, disponible)
        self.edicion= edicion
        self.periodicidad = periodicidad

    def obtenerEdicion(self):
        print(f"La edicion de la revista {self.titulo} es: {self.edicion}")

    def obtenerPeriodicidad(self):
        print(f"La revista '{self.titulo}' se publica de forma {self.periodicidad}")
```

Objetos:

```python
revist1= Revista(90, "Mensual", 18938490, "Algarabia", 2025, True )
revist2 = Revista(145, "Mensual", 18938491, "National Geographic", 2024, True)
revist3 = Revista(52, "Semanal", 18938492, "Time Magazine", 2024, True)
revist4 = Revista(12, "Anual", 18938493, "Resumen Científico", 2023, False)
revist5 = Revista(305, "Mensual", 18938494, "Muy Interesante", 2025, True)
revist6 = Revista(88, "Bimestral", 18938495, "Diseño Interior", 2024, True)
revist7 = Revista(15, "Semanal", 18938496, "The Economist", 2025, False)
revist8 = Revista(201, "Mensual", 18938497, "Vogue", 2024, True)
revist9 = Revista(45, "Trimestral", 18938498, "Arqueología Mexicana", 2023, True)
revist10 = Revista(10, "Semanal", 18938499, "Proceso", 2025, True)
```

Uso de métodos:

```python
revist1.obtenerEdicion()
print("\n")
revist1.obtenerPeriodicidad()
```

Terminal:

```
----Biblioteca Smart----

La edicion de la revista Algarabia es: 90


La revista 'Algarabia' se publica de forma Mensual
```

Clase MaterialDigital:

```python
class MaterialDigital(Material):
    def __init__(self, tipoArchivo, urlDescarga, tamañoMB, idMaterial, titulo, añoPublicacion, disponible):
        super(). __init__( idMaterial, titulo, añoPublicacion, disponible)
        self.tipoArchivo= tipoArchivo
        self.urlDescarga= urlDescarga
        self.tamañoMb= tamañoMB

    def descargar(self):
        if self.disponible:
            print(f"Descargando '{self.titulo}'...")
            print(f"Archivo: {self.tipoArchivo}")
            print(f"Tamaño: {self.tamañoMb} MB")
            print(f"URL: {self.urlDescarga}")
        else:
            print(f"El material '{self.titulo}' no está disponible para descarga.")

    def generarLink(self):
        print(f"Link de descarga para '{self.titulo}': {self.urlDescarga}")
```

Objetos:

```python
digi1 = MaterialDigital("PDF", "https://bit.ly/python-basico", 15.5, 300121, "Guía de Python 3", 2024, True)
digi2 = MaterialDigital("EPUB", "https://bit.ly/limpio-codigo", 4.2, 300231, "Código Limpio", 2008, True)
digi3 = MaterialDigital("PDF", "https://bit.ly/ia-futuro", 22.1, 300311, "Inteligencia Artificial", 2025, True)
digi4 = MaterialDigital("MP4", "https://bit.ly/curso-git", 450.0, 307485, "Video Curso Git & GitHub", 2023, True)
digi5 = MaterialDigital("PDF", "https://bit.ly/diseno-patrones", 18.3, 304635, "Patrones de Diseño", 1994, True)
digi6 = MaterialDigital("EPUB", "https://bit.ly/algoritmos-v3", 6.8, 30474, "Algoritmos Avanzados", 2022, True)
digi7 = MaterialDigital("PDF", "https://bit.ly/redes-cisco", 35.5, 3007377, "Fundamentos de Redes", 2024, True)
digi8 = MaterialDigital("MP4", "https://bit.ly/taller-docker", 310.0, 335735, "Workshop Docker Express", 2025, True)
digi9 = MaterialDigital("PDF", "https://bit.ly/sql-expert", 12.4, 305735, "SQL para Expertos", 2021, True)
digi10 = MaterialDigital("EPUB", "https://bit.ly/web-react", 5.9, 3035623, "React.js Moderno", 2024, True)
```

Uso de métodos:

```
digi10.descargar()
print("\n")
digi10.generarLink()
```

Terminal:

```
----Biblioteca Smart----

Descargando 'React.js Moderno'...
Archivo: EPUB
Tamaño: 5.9 MB
URL: https://bit.ly/web-react



Link de descarga para 'React.js Moderno': https://bit.ly/web-react
PS C:\Users\LATITUDE 5550>
```

Clase Persona:

```
class Persona():
    def __init__ (self, idPersona, name, email):
        self.idPersona= idPersona
        self.nombre = name
        self.email= email

    def log_in(self, nombre_ingresado, email_ingresado):
        print("Ingrese su usuario:""\n")
        if self.nombre == nombre_ingresado and self.email == email_ingresado:
           print(f"Usuario: {nombre_ingresado} /// Email: {email_ingresado}")
           print(f"---Bienvenido a Biblioteca Smart---""\n")

           if isinstance(self, Usuario):
               print(f"Has iniciado sesión {self.nombre}")

           elif isinstance(self, Bibliotecario):
               print(f"Has iniciado sesión como BIBLIOTECARIO con id: {self.idEmpleado}")

        else:
            print(f"Usuario: {nombre_ingresado} --- Email: {email_ingresado}")
            print("Correo o usuario incorrecto")

    def actualizar_perfil(self, nuevo_nombre, nuevo_email):
        print("Ingresa los nuevos datos:")
        self.nombre= nuevo_nombre
        self.email= nuevo_email
        print(f"Los datos han sido actualizados, el nuevo nombre es: {nuevo_nombre} y el nuevo email es: {nuevo_email} ")
```

Objetos:

```
per1= Persona(1738393, "Jetro", "jet@gmail.com")
per2= Persona (17389494, "Enisha", "Enis@gmail.com")
per3 = Persona(17389495, "Dante", "dante.al@gmail.com")
per4 = Persona(17389496, "Valeria", "val.rios@gmail.com")
per5 = Persona(17389497, "Julian", "july_99@hotmail.com")
per6 = Persona(17389498, "Beatriz", "bea.tris@outlook.com")
per7 = Persona(17389499, "Marcos", "m.perez@gmail.com")
per8 = Persona(17389500, "Elena", "ele.na88@yahoo.com")
per9 = Persona(17389501, "Samuel", "sam_pro@gmail.com")
per10 = Persona(17389502, "Fiona", "fiona_sky@gmail.com")
```

Uso de métodos:

```
per1.log_in("Jetro", "jet@gmail.com")
print("\n")
per1.actualizar_perfil("Jet", "jet@gmail.com")
print("\n")
#Prueba 2
per1.log_in("Jety", "jet@gmail.com")
```

Terminal:

```
----Biblioteca Smart----

Ingrese su usuario:

Usuario: Jetro /// Email: jet@gmail.com
---Bienvenido a Biblioteca Smart---



Ingresa los nuevos datos:
Los datos han sido actualizados, el nuevo nombre es: Jet y el nuevo email es: jet@gmail.com


Ingrese su usuario:

Usuario: Jety --- Email: jet@gmail.com
Correo o usuario incorrecto
PS C:\Users\LATITUDE 5550>
```

Clase Usuario:

```python
class Usuario(Persona):
    def __init__ (self, limitePrestamos, idPersona, name, email):
        super(). __init__(idPersona, name, email)
        self.limitePrestamos = limitePrestamos
        self.listaActiva= []
        self.bloqueado = False

    def consultarDisponibilidad(self, material):
        if material.disponible:
            print(f"El material '{material.titulo}' está disponible.")
            return True
        else:
            print(f"El material '{material.titulo}' no está disponible.")
            return False

    def solicitarPrestamo(self, material):
        print(f"{self.nombre} solicita el préstamo de '{material.titulo}'")
        return material
```

Objetos:

```python
usu1= Usuario(13, 124465, "Alice", "al@hjgmail.com")
usu2 = Usuario(13, 556677, "Mateo", "m.espinosa@outlook.com")
usu3 = Usuario(13, 882233, "Saori", "s.tanaka@workmail.jp")
usu4 = Usuario(13, 449911, "Thiago", "thiago.silva@brasil.br")
usu5 = Usuario(13, 773300, "Amira", "amira.h@global.net")
usu6 = Usuario(13, 225588, "Lukas", "l.muller@berlin.de")
usu7 = Usuario(13, 991144, "Ximena ", "xortiz@universidad.edu")
usu8 = Usuario(13, 336699, "Kofi ", "kmensah@accra.gh")
usu9 = Usuario(13, 660022, "Svetlana ", "s.ivanko@mail.ru")
usu10 = Usuario(13, 114477, "Chen ", "wei.chen@tech.cn")
```

Uso de métodos:

```python
usu1.consultarDisponibilidad(book1)
print("\n")
usu9.solicitarPrestamo(book1)
print("\n")
usu2.consultarDisponibilidad(book3)
```

Terminal:

```
----Biblioteca Smart----

El material 'Inferno' está disponible.


Svetlana  solicita el préstamo de 'Inferno'


El material '1984' no está disponible.
PS C:\Users\LATITUDE 5550>
```

Clase Bibliotecario:

```python
class Bibliotecario(Persona):
    def __init__ (self, idEmpleado, idPersona, name,  email):
        super(). __init__ (idPersona, name, email)
        self.idEmpleado= idEmpleado

    def gestionarPrestamo(self, usuario, material, idPrestamo ):
        if isinstance(usuario, Usuario):
            if material.disponible:
                if len(usuario.listaActiva) < usuario.limitePrestamos:
                    prestamo = Prestamo(idPrestamo, usuario, material, datetime.now(), datetime.now() + timedelta(days=7))
                    usuario.listaActiva.append(prestamo)
                    material.actualizarDisponibilidad()

                    print(f"Préstamo registrado por el bibliotecario {self.nombre}")
                else :
                    print("El usuario alcanzó su límite de préstamos")
            else:
                print("El material no está disponible")
        else:
            print("Solo se pueden prestar materiales a usuarios")

    def transferirMaterial(self, material, sucursalOrigen, sucursalDestino):
        if sucursalOrigen.verificarStock(material) and material.disponible:
            sucursalOrigen.catalogoLocal.remove(material)
            sucursalDestino.catalogoLocal.append(material)
            print(f"El material '{material.titulo}' fue transferido de {sucursalOrigen.nombreSucursal} a {sucursalDestino.nombreSucursal}")

        else:
            print(f"El material '{material.titulo}' no existe en el catálogo de {sucursalOrigen.nombreSucursal}")
```

Objetos:

```
biblio1= Bibliotecario(12243545, 2312441, "Juan Carlos", "juanw@gmail.com")
biblio2 = Bibliotecario(12243546, 2312442, "Tulio", "tulio.triv@biblioteca.com")
biblio3 = Bibliotecario(12243547, 2312443, "Juanín", "juanin.j@correo.cl")
biblio4 = Bibliotecario(12243548, 2312444, "Guaripolo", "guaripolo_favorito@oficina.com")
biblio5 = Bibliotecario(12243549, 2312445, "Patana", "patana.t@prensa.cl")
biblio6 = Bibliotecario(12243550, 2312446, "Mario Hugo", "mhugo.perros@mascotas.com")
biblio7 = Bibliotecario(12243551, 2312447, "Policarpo", "policarpo.top@ranking.cl")
biblio8 = Bibliotecario(12243552, 2312448, "Mico el Micófono", "mico.mic@entrevistas.com")
biblio9 = Bibliotecario(12243553, 2312449, "Huachimingo", "huachi.coleccion@peluches.cl")
biblio10 = Bibliotecario(12243554, 2312450, "Calcetín", "calcetin.rombos@heroes.com")
```

Uso de métodos:

```
store1.actualizarCatalogo(book1)
print("\n")
store1.verificarStock(book1)

biblio10.gestionarPrestamo("Julian", book2, 13124252)
print("\n")
biblio10.gestionarPrestamo(usu5, book2, 13124252)
print("\n")
biblio10.transferirMaterial(book1,store1, store3)
```

Terminal:

```
----Biblioteca Smart----

Se agregó un nuevo material: Inferno


El material Inferno se encuentra disponible
Solo se pueden prestar materiales a usuarios


Disponibilidad de 'Cien años de soledad' actualizada a: False
Préstamo registrado por el bibliotecario Calcetín


El material Inferno se encuentra disponible
El material 'Inferno' fue transferido de Biblioteca del norte a Biblioteca del Sur
PS C:\Users\LATITUDE 5550>
```

Clase Sucursal:

```python
class Sucursal():
    def __init__ (self, idSucursal, nombreSucursal):
        self.idSucursal = idSucursal
        self.nombreSucursal = nombreSucursal
        self.catalogoLocal= []

    def actualizarCatalogo(self, material):
        self.catalogoLocal.append(material)
        print(f"Se agregó un nuevo material: {material.titulo}")

    def verificarStock(self, material):
        if material in self.catalogoLocal:
            print(f"El material {material.titulo} se encuentra disponible")
            return True
        else:
            print (f"Material no disponible")
            return False
```

Objetos:

```python
store1= Sucursal(137484, "Biblioteca del norte")
store2 =  Sucursal(137485, "Sede Central Histórica")
store3 =  Sucursal(137486, "Biblioteca del Sur ")
store4 =  Sucursal(137487, "Sucursal Oriente - Parque")
store5 =  Sucursal(137488, "Biblio-Tec Poniente")
store6 =  Sucursal(137489, "Sede Digital Metropolitana")
store7 =  Sucursal(137490, "Biblioteca Infantil Los Pinos")
store8 =  Sucursal(137491, "Sucursal Universitaria")
store9 =  Sucursal(137492, "Estación Cultural del Este")
store10 = Sucursal(137493, "Biblio-Móvil Itinerante")
```

Uso de métodos:

```python
store1.actualizarCatalogo(book10)
print("\n")
store1.verificarStock(book10)
```

Terminal:

```
----Biblioteca Smart----

Se agregó un nuevo material: El viejo y el mar


El material El viejo y el mar se encuentra disponible
PS C:\Users\LATITUDE 5550>
```

Clase Prestamo:

```python
class Prestamo():
    def __init__(self, idPrestamo, usuario, material, fechaInicio, fechaDevolucion):
        self.idPrestamo = idPrestamo
        self.usuario = usuario
        self.material = material
        self.fechaInicio = fechaInicio
        self.fechaDevolucion = fechaDevolucion
        self.activo = True

    def finalizarPrestamo(self):
        if self.activo:
            self.activo = False
            self.material.disponible = True

            if self in self.usuario.listaActiva:
                self.usuario.listaActiva.remove(self)

            print(f"El préstamo {self.idPrestamo} ha finalizado.")
            print(f"El usuario {self.usuario.nombre} devolvió '{self.material.titulo}'.")
            print(f"El material ahora está disponible nuevamente.")

        else:
            print("El préstamo ya estaba finalizado")


    def estaVencido(self):

        fecha_actual = datetime.now()

        if fecha_actual > self.fechaDevolucion:
            print(f"El préstamo {self.idPrestamo} está vencido")
            return True
        else:
            print(f"El préstamo {self.idPrestamo} aún está en tiempo")
            return False
```

Objetos:

```
presta1= Prestamo(1283873, usu3, book3, datetime.now(), datetime.now() + timedelta(days=7))
presta2 = Prestamo(1283874, usu1, book1, datetime.now(), datetime.now() + timedelta(days=-20))
presta3 = Prestamo(1283875, usu2, digi3, datetime.now(), datetime.now() + timedelta(days=7))
presta4 = Prestamo(1283876, usu4, revist1, datetime.now(), datetime.now() + timedelta(days=5))
presta5 = Prestamo(1283877, usu5, mat8, datetime.now(), datetime.now() + timedelta(days=7))
presta6 = Prestamo(1283878, usu6, revist2, datetime.now(), datetime.now() + timedelta(days=5))
presta7 = Prestamo(1283879, usu7, book5, datetime.now(), datetime.now() + timedelta(days=7))
presta8 = Prestamo(1283880, usu8, revist3, datetime.now(), datetime.now() + timedelta(days=5))
presta9 = Prestamo(1283881, usu9, digi4, datetime.now(), datetime.now() + timedelta(days=7))
presta10 = Prestamo(1283882, usu10, mat2, datetime.now(), datetime.now() + timedelta(days=7))
```

Uso de métodos:

```
presta1.finalizarPrestamo()
presta1.estaVencido()
```

Terminal:

```
----Biblioteca Smart----

El préstamo 1283873 ha finalizado.
El usuario Saori devolvió '1984'.
El material ahora está disponible nuevamente.


El préstamo 1283873 aún está en tiempo
PS C:\Users\LATITUDE 5550>
```

Clase Penalizacion:

```python
class Penalizacion ():
    def __init__(self, monto, motivo, pagada):
        self.monto= monto
        self.motivo= motivo
        self.pagada= pagada

    def calcularMulta(self, prestamo):
        if prestamo.estaVencido():
            dias_retraso = (datetime.now() - prestamo.fechaDevolucion).days
            self.monto = dias_retraso * 10
            self.motivo = f"Retraso de {dias_retraso} días"

            print(f"Multa generada: ${self.monto}")
            print(f"---Motivo: {self.motivo}---")
            return self.monto

        else:
            print("No hay multa, el préstamo no está vencido")
            return 0

    def bloquearUsuario(self, usuario):
        if not self.pagada:
            if not usuario.bloqueado:
                usuario.bloqueado = True
                print(f"El usuario {usuario.nombre} ha sido bloqueado por falta de pago")
            else:
                print("El usuario ya esta bloqueado")
```

Objetos:

```python
pena1 = Penalizacion(0, "Sin motivo", False)
pena2 = Penalizacion(50, "Retraso de 5 días", True)
pena3 = Penalizacion(10, "Retraso de 1 día", True)
pena4 = Penalizacion(20, "Retraso de 2 días", False)
pena5 = Penalizacion(30, "Retraso de 3 días", False)
pena6 = Penalizacion(70, "Retraso de 7 días", True)
pena7 = Penalizacion(100, "Retraso de 10 días", False)
pena8 = Penalizacion(150, "Retraso de 15 días", False)
pena9 = Penalizacion(40, "Retraso de 4 días", True)
pena10 = Penalizacion(300, "Retraso de 30 días", False)
```

Uso de métodos:

```python
pena10.calcularMulta(presta2)
print("\n")
pena10.bloquearUsuario(usu4)
print("\n")
pena7.calcularMulta(presta2)
```

Terminal:

```
----Biblioteca Smart----

El préstamo 1283874 está vencido
Multa generada: $200
---Motivo: Retraso de 20 días---


El usuario Thiago ha sido bloqueado por falta de pago


El préstamo 1283874 está vencido
Multa generada: $200
---Motivo: Retraso de 20 días---
PS C:\Users\LATITUDE 5550>
```

Clase Catalogo:

```python
class Catalogo():
    def __init__ (self):
        self.materiales= []

    def agregarMaterial(self, material):
        self.materiales.append(material)

    def buscarPorAutor(self, autor):
        resultados = []

        for material in self.materiales:
            if isinstance(material, Libro) and material.autor == autor:
                resultados.append(material)

        if resultados:
            for u in resultados:
                print(f"Encontrado: {u.titulo}")
        else:
            print("No se encontraron libros de ese autor")


    def buscarEnSucursales(self, material, sucursales):
        if not isinstance(sucursales, list):
            sucursales= [sucursales]

        encontrado= False
        for sucursal in sucursales:
            if material in sucursal.catalogoLocal:
                estado = "Disponible" if material.disponible else "No disponible"
                print(f"{material.titulo} -> {sucursal.nombreSucursal} ({estado})")
                encontrado = True

        if not encontrado:
            print("El material no se encuentra en ninguna sucursal")
```

Objetos:

```
cata1= Catalogo()
```

Uso de métodos:

```
store1.actualizarCatalogo(book1)


cata1.agregarMaterial(book1)
print("\n")
cata1.buscarPorAutor("Dan Brown")
print("\n")
cata1.buscarEnSucursales(book1, store1)
```

Terminal:

```
----Biblioteca Smart----

Se agregó un nuevo material: Inferno


Encontrado: Inferno


Inferno -> Biblioteca del norte (Disponible)
PS C:\Users\LATITUDE 5550>
```